Task 3: Product Detail, Persistent Cart, and Save-for-Later

Phase 2, Task 3 deliverable (revision 2, after review). Author: Ahmed. Date: 2026-11-12. Branch: `feature/product-detail-cart-save-for-later`.

Try it: `npm install`, `npm run start`, open http://127.0.0.1:8080/catalog.html, choose a product, add it to the cart and open **Cart** in the menu. `product.html?id=usb-c-hub` is out of stock; `product.html?id=desk-lamp` has only 3 left. **Run the tests:** `npm test` (Vitest) or, with nothing installed, `npm run test:node`.

1. Changes After Review

Review point	What changed	Evidence
Totals, shipping and currency match reference; badge = page	Fixtures now also hold the reference display strings (`$1,999.90`), checked against `formatMoney`. The badge and the cart page both use one function, `cartSummary()`, so they cannot disagree (before, an out-of-stock line was counted by the badge but not by the page)	unit: 8 fixture cases × 3 amounts × (value + format); e2e "badge always equals the page"
Out of stock explains itself; quantity bounds (0, > stock, > limit)	New pure `validateQuantity()` returns the corrected quantity **and the reason**, shown next to the stepper: "The smallest quantity is 1.", "Only 3 in stock.", "The limit is 10 per item." Setting a cart line to 0 removes it; adding 0 is refused with "Choose a quantity of at least 1."	unit "quantity bounds" (5 tests); e2e "quantity bounds"
Unit/integration tests pass	Tests now run in Node by default (no DOM needed); storage is injected (`createStorage`) instead of patching `window`. New `npm run test:node` runs the **same files** with Node's built-in runner and a small Vitest-compatible shim committed in `tests/support/`, so they pass with zero installed packages	`npm run test:node`: **134 of 134** (109 unit, 25 integration)
Cart/reducer logic is pure (no DOM, storage or network inside)	`state/cart.js`, `state/saved.js`, `state/recent.js` now contain only pure functions (+ `cartReducer`). All storage moved to `state/stores.js`. ESLint forbids `window`, `document`, `localStorage`, `sessionStorage`, `fetch`, `XMLHttpRequest`, `navigator`, `location`, `history`, `Date` and storage/DOM/API imports in those files	`npm run lint` (probe file reported); unit tests pass deep-frozen states
Persist as specified and survive corrupted data	Every stored value is repaired on load (`rehydrate`, `rehydrateSaved`, `rehydrateRecent`): bad entries dropped, duplicates merged, numeric strings accepted, missing image/currency defaulted, quantities re-clamped; the repaired value is written back once. Corrupt JSON or blocked storage never throws	unit "corrupted stored data" (4 cases); e2e "corrupted … never breaks a page" on 4 pages
Requests match the API specification	The spec now declares the optional `X-Client-Id` header (sent by the Axios module) and the mock-only test headers. Every `/api` request made during the whole e2e run is checked against `api/openapi.json` (path, method, query parameters and values, path parameters, `X-` headers)	e2e: **197 requests checked, 0 problems**; `npm run contract`: 50 requests, 0 problems

2. What Was Built

Requirement	Where
Product detail: image gallery, specs table, stock status, quantity stepper, add to cart	`components/product-detail.js`, `gallery.js`, `quantity-stepper.js`
Cart as pure functions over a state object (add, update quantity, remove, clear) + reducer	`state/cart.js`: `addToCart`, `updateQuantity`, `removeFromCart`, `clearCart`, `canAdd`, `cartReducer`, `cartSummary`
Totals, shipping rule	`utils/cart-math.js`: `totals`, `shippingFor`, `validateQuantity`
One currency format (`Intl.NumberFormat`)	`utils/money.js` `formatMoney`, used for every price on every page
Cart in localStorage, rehydrated on load	`state/stores.js` `createCartStore` + `state/cart.js` `rehydrate`
Save-for-later (add, move to cart, remove), persisted separately	`state/saved.js` (pure) + `createSavedStore`
Recently viewed in sessionStorage, rendered as a strip	`state/recent.js` (pure) + `createRecentStore`, `components/recent-strip.js`
Destructuring and spread in every state update	all functions in `state/cart.js`, `saved.js`, `recent.js`
One API module with Axios and a request interceptor	`api/product-api.js` (Axios 1.19.0 vendored in `src/vendor/axios.js`)

3. Totals: Rule and Fixture Check

Free shipping from a **$100.00** subtotal, otherwise **$7.99**; empty cart: no shipping; no tax. Money is integer cents; display only through `formatMoney`. Out-of-stock lines stay visible but are neither counted nor charged.

`tests/fixtures/cart-cases.json`: reference totals computed separately with Python's `Decimal` (dollars) plus the expected display strings. Tolerance **$0.005**.

Fixture case	Subtotal	Shipping	Total
empty cart	$0.00	$0.00	$0.00
one item below free shipping	$24.00	$7.99	$31.99
quantity 3 of one item	$89.97	$7.99	$97.96
just below the $100 threshold	$98.99	$7.99	$106.98
just above the $100 threshold	$101.00	$0.00	$101.00
two lines, one with quantity 2	$499.97	$0.00	$499.97
maximum quantity on one line (10)	$1,999.90	$0.00	$1,999.90
four lines, one at its stock limit	$499.48	$0.00	$499.48

4. Quantity Rules and Out-of-Stock Behaviour

Request	Product page	Cart line	Pure function result
0 or less	set to 1, "The smallest quantity is 1."	line removed	`updateQuantity(…, 0)` removes; `addToCart(…, 0)` refused: "Choose a quantity of at least 1."
More than the stock (desk lamp: 3)	set to 3, "Only 3 in stock."	set to 3, same message	clamped to the stock
More than 10	set to 10, "The limit is 10 per item."	set to 10, same message	clamped to 10
Not a number	set to 1, "Enter a whole number of items."	same	—
Out of stock	"Out of stock"; Add to cart disabled and linked (`aria-describedby`) to "This item is out of stock, so it cannot be added to the cart…"	"Out of stock: not included in the total…"	`canAdd` → `{ ok: false, reason }`; `addToCart` returns the unchanged state
Saved item out of stock	—	"Move to cart" disabled, reason shown	`moveToCart` returns both lists unchanged + reason

5. Persistence and Corrupted Data

Data	Storage	Key	Refresh	Tab close / return visit	New tab
Cart	localStorage	`brandstore:cart:v1`	kept	kept	shared
Saved for later	localStorage (separate key)	`brandstore:saved:v1`	kept	kept	shared
Recently viewed (max 6, newest first, no duplicates)	sessionStorage	`brandstore:recent:v1`	kept	cleared	not shared

Stored data	What happens on load
Not JSON (`{not json`) / wrong type (42, `"text"`, `{ lines: "x" }`)	empty list
Line without id, name or a valid price; quantity 0 or negative	dropped
Same product twice	merged into the first
Quantity as text (`"3"`), above stock or above 10	converted and re-clamped

Stored data	What happens on load
Missing image or bad currency	placeholder image, `USD`
Storage blocked (private mode) or full	the page works in memory; nothing throws

The repaired value is written back once, so the corruption is cleaned up.

6. Axios in One Module, fetch Everywhere Else

`src/api/product-api.js` (product page and the cart's price/stock re-check, `GET /products/{id}`) is the only Axios module. Its **request interceptor** adds `Accept: application/json` and `X-Client-Id` (declared in the spec) and stamps the start time; its **response interceptor** records the duration and converts every failure into the same `ApiError` the fetch modules throw. The rest of the app uses `fetch` (`src/api/http.js`): built in (0 KB against 48 KB), native `AbortController` for the catalog's cancelled searches, and nothing Axios adds is needed there. Axios is copied as an ES module from the npm package (`npm run vendor`, version pinned) because there is no bundler.

7. QA Matrix

Journey (`npm run e2e`)	Chrome desktop 1280	Chrome phone 375	Firefox	Safari / WebKit
1. Gallery, specs table, stock status; stepper limited by stock	Pass	Pass	Not run	Not run
2. Out-of-stock product cannot be added and explains why	Pass	Pass	Not run	Not run
3. Totals match the fixtures just below and above free shipping	Pass	Pass	Not run	Not run
4. Cart survives refresh and a return visit; + updates the total	Pass	Pass	Not run	Not run
5. Save for later and back; lists persist separately; clear cart	Pass	Pass	Not run	Not run
6. Recently viewed: no duplicates, capped at 6, not shared with a new tab	Pass	Pass	Not run	Not run
7. Stale stored price corrected from the API and announced	Pass	Pass	Not run	Not run
8. One price format ($1,234.56) on catalog, product, cart and home pages	Pass	Pass	Not run	Not run
9. Quantity bounds: 0, above stock, above 10; typing 0 in the cart removes the line	Pass	Pass	Not run	Not run
10. Menu badge equals the page's item count, also with an out-of-stock line	Pass	Pass	Not run	Not run
11. Corrupted cart, saved and recent data on 4 pages	Pass	Pass	Not run	Not run
Catalog and Phase 1 journeys (18; one desktop-only, one phone-only)	17 of 17	17 of 17	Not run	Not run
Every `/api` request of the run checked against `api/openapi.json`	197 requests, 0 problems			

Total: **56 of 56 journey runs pass** in Chromium (and the run fails if any request breaks the spec).

Other check	Result
`npm run test:node` (same files as `npm test`)	134 of 134 (109 unit, 25 integration against the real mock server, the Netlify function and Axios)
`npm run lint` / `npm run format:check`	0 problems / all formatted
`npm run contract`	50 requests, 0 problems
`npm run build`	OK: 54 modules reachable from 6 pages
`node qa/js-rules.js`	0 failures (60 files, 193 functions, all ≤ 40 lines)
`node qa/preflight.js` / `node qa/headers.js` / `node qa/cls.js`	322 of 322 / 0 failures / layout shift 0

8. Deviations List

#	Deviation	Reason
1	Axios is vendored into `src/vendor/axios.js`	No bundler; browsers cannot resolve `import 'axios'`. `npm run vendor` refreshes it
2	Tests were run here with `npm run test:node` (Node's runner + the committed shim), not with Vitest	Vitest could not be installed in the build environment; `npm test` runs the same files with Vitest
3	MSW not used	Integration tests call the real mock server instead
4	No tax	Not in the brief or the API
5	Gallery images 2 and 3 are generic placeholders	The catalog has one photo per product
6	Checkout button disabled	Checkout is a later task
7	Firefox, Safari and real devices not tested	Not available in the build environment